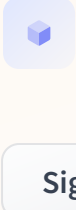
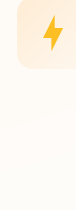
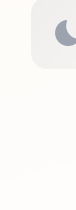


Join the Hugging Face community

and get access to the augmented documentation experience

 Collaborate on models, datasets and Spaces

 Faster examples with accelerated inference

 Switch between documentation themes

Sign Up

to get started

A full training

 Ask a question Open in Colab Open in Studio Lab

Write your training loop in PyTorch



Now we'll see how to achieve the same results as we did in the last section without using the `Trainer` class. Again, we assume you have done the data processing in section 2. Here is a short summary covering everything you will need:

```
from datasets import load_dataset
from transformers import AutoTokenizer, DataCollatorWithPadding

raw_datasets = load_dataset("glue", "mrpc")
checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

def tokenize_function(example):
    return tokenizer(example["sentence1"], example["sentence2"], truncation=True)

tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

Prepare for training

Before actually writing our training loop, we will need to define a few objects. The first ones are the dataloaders we will use to iterate over batches. But before we can define those dataloaders, we need to apply a bit of postprocessing to our `tokenized_datasets`, to take care of some things that the `Trainer` did for us automatically. Specifically, we need to:

- Remove the columns corresponding to values the model does not expect (like the `sentence1` and `sentence2` columns).
- Rename the column `label` to `labels` (because the model expects the argument to be named `labels`).
- Set the format of the datasets so they return PyTorch tensors instead of lists.

Our `tokenized_datasets` has one method for each of those steps:

```
tokenized_datasets = tokenized_datasets.remove_columns(["sentence1", "sentence2", "idx"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format("torch")
tokenized_datasets["train"].column_names
```

We can then check that the result only has columns that our model will accept:

```
["attention_mask", "input_ids", "labels", "token_type_ids"]
```

Now that this is done, we can easily define our dataloaders:

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(
    tokenized_datasets["train"], shuffle=True, batch_size=8, collate_fn=data_collator
)
eval_dataloader = DataLoader(
    tokenized_datasets["validation"], batch_size=8, collate_fn=data_collator
)
```

To quickly check there is no mistake in the data processing, we can inspect a batch like this:

```
for batch in train_dataloader:
    break

{k: v.shape for k, v in batch.items()}

{'attention_mask': torch.Size([8, 65]),
 'input_ids': torch.Size([8, 65]),
 'labels': torch.Size([8]),
 'token_type_ids': torch.Size([8, 65])}
```

Note that the actual shapes will probably be slightly different for you since we set `shuffle=True` for the training dataloader and we are padding to the maximum length inside the batch.

Now that we're completely finished with data preprocessing (a satisfying yet elusive goal for any ML practitioner), let's turn to the model. We instantiate it exactly as we did in the previous section:

```
from transformers import AutoModelForSequenceClassification

model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
```

To make sure that everything will go smoothly during training, we pass our batch to this model:

```
outputs = model(*batch)
print(outputs.loss, outputs.logits.shape)
```

```
tensor(0.5441, grad_fn=<NllLossBackward>) torch.Size([8, 2])
```

All 🤖 Transformers models will return the loss when `labels` are provided, and we also get the logits (two for each input in our batch, so a tensor of size `8 x 2`).

We're almost ready to write our training loop! We're just missing two things: an optimizer and a learning rate scheduler. Since we are trying to replicate what the `Trainer` was doing by hand, we will use the same defaults. The optimizer used by the `Trainer` is `AdamW`, which is the same as `Adam`, but with a twist for weight decay regularization (see "[Decoupled Weight Decay Regularization](#)" by Ilya Loshchilov and Frank Hutter):

```
from transformers import AdamW

optimizer = AdamW(model.parameters(), lr=5e-5)
```

Finally, the learning rate scheduler used by default is just a linear decay from the maximum value (`5e-5`) to `0`. To properly define it, we need to know the number of training steps we will take, which is the number of epochs we want to run multiplied by the number of training batches (which is the length of our training dataloader). The `Trainer` uses three epochs by default, so we will follow that:

```
from transformers import get_scheduler

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
print(num_training_steps)
```

```
1377
```

The training loop

One last thing: we will want to use the GPU if we have access to one (on a CPU, training might take several hours instead of a couple of minutes). To do this, we define a `device` we will put our model and our batches on:

```
import torch

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
model.to(device)
device

device(type='cuda')
```

We are now ready to train! To get some sense of when training will be finished, we add a progress bar over our number of training steps, using the `tqdm` library:

```
from tqdm.auto import tqdm

progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(*batch)
        loss = outputs.loss
        loss.backward()

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

You can see that the core of the training loop looks a lot like the one in the introduction. We didn't ask for any reporting, so this training loop will not tell us anything about how the model fares. We need to add an evaluation loop for that.

The evaluation loop

As we did earlier, we will use a metric provided by the 🤖 Evaluate library. We've already seen the `metric.compute()` method, but metrics can actually accumulate batches for us as we go over the prediction loop with the method `add_batch()`. Once we have accumulated all the batches, we can get the final result with `metric.compute()`. Here's how to implement all of this in an evaluation loop:

```
import evaluate

metric = evaluate.load("glue", "mrpc")
model.eval()
for batch in eval_dataloader:
    batch = {k: v.to(device) for k, v in batch.items()}
    with torch.no_grad():
        outputs = model(*batch)

    logits = outputs.logits
    predictions = torch.argmax(logits, dim=-1)
    metric.add_batch(predictions=predictions, references=batch["labels"])

metric.compute()
```

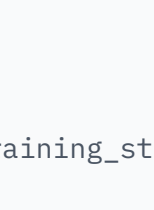
```
{'accuracy': 0.8431372549019608, 'f1': 0.8907849829351535}
```

Again, your results will be slightly different because of the randomness in the model head initialization and the data shuffling, but they should be in the same ballpark.

 **Try it out!** Modify the previous training loop to fine-tune your model on the SST-2 dataset.

Supercharge your training loop with 🚀 Accelerate

Supercharge your PyTorch training loop with Accelerate



The training loop we defined earlier works fine on a single CPU or GPU. But using the 🚀 [Accelerate](#) library, with just a few adjustments we can enable distributed training on multiple GPUs or TPUs. Starting from the creation of the training and validation dataloaders, here is what our manual training loop looks like:

```
from transformers import AdamW, AutoModelForSequenceClassification, get_scheduler

model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
optimizer = AdamW(model.parameters(), lr=3e-5)

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
model.to(device)

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)

progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(*batch)
        loss = outputs.loss
        loss.backward()

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

And here are the changes:

```
+ from accelerate import Accelerator
from transformers import AdamW, AutoModelForSequenceClassification, get_scheduler

+ accelerator = Accelerator()

model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
optimizer = AdamW(model.parameters(), lr=3e-5)

- device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
+ model.to(device)

+ train_dataloader, eval_dataloader, model, optimizer = accelerator.prepare(
+     train_dataloader, eval_dataloader, model, optimizer
+ )

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)

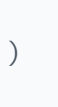
progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(*batch)
        loss = outputs.loss
        + loss.backward()
        + accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

The first line to add is the import line. The second line instantiates an `Accelerator` object that will look at the environment and initialize the proper distributed setup. 🚀 `Accelerate` handles the device placement for you, so you can remove the lines that put the model on the device (or, if you prefer, change them to use `accelerator.device` instead of `device`).

Then the main bulk of the work is done in the line that sends the dataloaders, the model, and the optimizer to `accelerator.prepare()`. This will wrap those objects in the proper container to make sure your distributed training works as intended. The remaining changes to make are removing the line that puts the batch on the device (again, if you want to keep this you can just change it to use `accelerator.device`) and replacing `loss.backward()` with `accelerator.backward(loss)`.

 **In order to benefit from the speed-up offered by Cloud TPUs, we recommend padding your samples to a fixed length with the "padding='max_length'" and "max_length" arguments of the tokenizer.**

If you'd like to copy and paste it to play around, here's what the complete training loop looks like with 🚀 `Accelerate`:

```
from accelerate import Accelerator
from transformers import AdamW, AutoModelForSequenceClassification, get_scheduler

accelerator = Accelerator()

model = AutoModelForSequenceClassification.from_pretrained(checkpoint, num_labels=2)
optimizer = AdamW(model.parameters(), lr=3e-5)

train_dataloader, eval_dataloader, model, optimizer = accelerator.prepare(
    train_dataloader, eval_dataloader, model, optimizer
)

num_epochs = 3
num_training_steps = num_epochs * len(train_dataloader)
lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)

progress_bar = tqdm(range(num_training_steps))

model.train()
for epoch in range(num_epochs):
    for batch in train_dataloader:
        outputs = model(*batch)
        loss = outputs.loss
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)
```

Putting this in a `train.py` script will make that script runnable on any kind of distributed setup. To try it out in your distributed setup, run the command:

```
accelerate config
```

which will prompt you to answer a few questions and dump your answers in a configuration file used by this command:

```
accelerate launch train.py
```

which will launch the distributed training.

If you want to try this in a Notebook (for instance, to test it with TPUs on Colab), just paste the code in a `training_function()` and run a last cell with:

```
from accelerate import notebook_launcher

notebook_launcher(training_function)
```

You can find more examples in the 🚀 [Accelerate repo](#).

